САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЁТ

ПО ДИСЦИПЛИНЕ «БЭК-ЭНД РАЗРАБОТКА»

по теме:

ДЗ6: НАСТРОЙКА GITHUB ACTIONS ДЛЯ АВТОМАТИЧЕСКОГО РАЗВЁРТЫВАНИЯ БЭКЕНД-ПРИЛОЖЕНИЯ

Проверил: Добряков Д. И.

_____

подпись, дата

Выполнил: Субботин Александр, БР1.1

_____

подпись, дата

Санкт-Петербург 2026

1 Задача

Настроить автодеплой приложения на удалённый сервер с триггером на обновление кода в репозитории на определённой ветке, используя GitHub Actions.

Разворачивается сервис аренды недвижимости из ЛР4–5 (`labs/lab4-5`) на сервер `13.143.182.131`, подготовленный скриптом `bootstrap.sh` из ЛР4. Workflow: `.github/workflows/rental-cicd.yml`.

2 Ход работы

2.1 Схема пайплайна

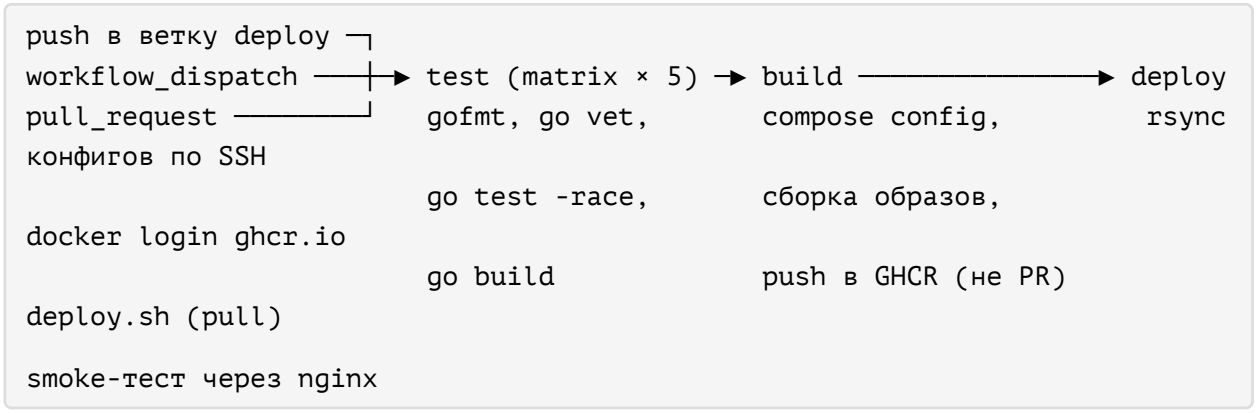


Таблица 1. Триггеры workflow

Событие	Что выполняется
`push` в `deploy`	test → build → deploy
`workflow_dispatch`	test → build → deploy (ручной запуск из вкладки Actions)
`pull_request`	только test и build — проверка изменений до слияния

Workflow запускается, только если изменились файлы проекта (`paths: "БР1.1/Субботин Александр/labs/lab4-5/**"`) или сам workflow — в репозитории лежат и другие работы, и их изменения не должны вызывать деплой.

2.2 Job test: проверка кода

Каждый микросервис — отдельный Go-модуль, поэтому используется matrix из пяти сервисов (`fail-fast: false` — видно все упавшие сервисы сразу). Версия Go берётся из `go.mod` сервиса, кэш модулей — по его `go.sum`:

```yaml
test:
  strategy:
    fail-fast: false
```

```
    matrix:
      service: [user-service, property-service, booking-service, chat-
service, api-gateway]
  steps:
    - uses: actions/checkout@v6
    - uses: actions/setup-go@v6
      with:
        go-version-file: ${{ env.APP_DIR }}/${{ matrix.service }}/go.mod
        cache-dependency-path: ${{ env.APP_DIR }}/${{ matrix.service }}/
go.sum
    - name: gofmt
      run: test -z "$(gofmt -l .)"
    - run: go vet ./...
    - run: go test -race -count=1 ./...
    - run: go build -o /dev/null ./src/service
```

Для CI добавлены unit-тесты: маппинг gRPC-кодов в HTTP-статусы и передача заголовка `Authorization` в gateway, очистка необязательных полей объявления в property-service, recovery паник и классификация ошибок в Sentry-интерцепторе property/booking.

2.3 Job build: сборка образов

```
build:
  needs: test
  permissions: { contents: read, packages: write }
  steps:
    - uses: actions/checkout@v6
    - name: имена образов           # GHCR требует нижний регистр
      run: |
        echo "IMAGE_PREFIX=ghcr.io/${GITHUB_REPOSITORY_OWNER,,}/rental" >>
"$GITHUB_ENV"
        echo "IMAGE_TAG=${GITHUB_SHA::12}" >> "$GITHUB_ENV"
    - run: |
        cp .env.example .env
        docker compose config --quiet      # валидность compose-файлов и
переменных
    - run: docker compose build --pull     # сборка всех Dockerfile
    - name: push в GHCR
      if: github.event_name != 'pull_request'
      run: |
        printf '%s' "$GHCR_TOKEN" | docker login ghcr.io -u "$GITHUB_ACTOR"
--password-stdin
        docker compose push user-service property-service booking-service
chat-service api-gateway
```

Так ошибка в Dockerfile или compose-файле обнаруживается до того, как попадёт на сервер, а сервер получает готовые образы `ghcr.io/zzisst/backend-rental-<сервис>:<sha>`. Сборка в CI выбрана потому, что ВМ общая с VPN и слабая (1 vCPU, 1.9 ГБ RAM): компиляция пяти Go-сервисов на ней занимала бы минуты и вытесняла бы VPN в swap. Авторизация в реестре — встроенным `GITHUB_TOKEN` (`packages: write`), отдельные токены не нужны.

2.4 Job deploy: доставка на сервер

```
deploy:
  needs: build
  if: github.event_name == 'workflow_dispatch' ||
      (github.event_name == 'push' && github.ref == 'refs/heads/deploy')
  environment: { name: production, url: ${{ vars.PUBLIC_URL }} }
  concurrency: { group: rental-production, cancel-in-progress: false }
  steps:
    - uses: actions/checkout@v6
    - name: SSH-ключ           # ключ и known_hosts из secrets, alias
"target" в ~/.ssh/config
    - name: rsync проекта на сервер
      run: rsync -az --delete --exclude '.env' --exclude 'docs/'
"$APP_DIR/" "target:$DEPLOY_PATH/"
    - name: nginx (если задан NGINX_SERVER_NAME)
      run: ssh target "cd $DEPLOY_PATH && ./deploy/scripts/setup-nginx.sh
$NGINX_SERVER_NAME"
    - name: docker login ghcr.io на сервере
      run: printf '%s' "$GHCR_TOKEN" | ssh target "docker login ghcr.io -u
$GITHUB_ACTOR --password-stdin"
    - name: docker compose up
      run: ssh target "cd $DEPLOY_PATH && DEPLOY_MODE=pull
IMAGE_PREFIX=$IMAGE_PREFIX \
             IMAGE_TAG=$IMAGE_TAG SENTRY_RELEASE=$IMAGE_TAG ./deploy/
scripts/deploy.sh"
    - name: docker logout
      if: always()
      run: ssh target "docker logout ghcr.io"
    - name: smoke-тест через nginx
      run: |
        curl -fsS --retry 10 --retry-delay 3 --retry-all-errors
"$PUBLIC_URL/health"
        curl -fsS "$PUBLIC_URL/api/v1/properties?limit=1" > /dev/null
        curl -fsS -o /dev/null "$PUBLIC_URL/grafana/api/health"
```

Решения:

- **образы — через GHCR, конфиги — через rsync**: на сервер копируются только compose-файлы, конфиги мониторинга и скрипты; `--delete`

удаляет с сервера файлы, удалённые из репозитория, а `--exclude .env` защищает серверные секреты (исключённые файлы rsync не удаляет);

- **токен реестра живёт только во время job'а**: `GITHUB_TOKEN` передаётся на сервер через stdin для `docker login` и после деплоя выполняется `docker logout`; на сервере не остаётся долгоживущих учётных данных;
- **тег образа = короткий sha коммита** и записывается в `.env` на сервере — видно, какая версия развёрнута, а откат — это `IMAGE_TAG=<старый sha> ./deploy/scripts/deploy.sh`;
- **`deploy.sh`** из ЛР4 общий для ручного и автоматического деплоя; он ждёт `healthy` у всех контейнеров и завершается с ошибкой (и логами), если сервис не поднялся, — тогда job становится красным;
- **`SENTRY_RELEASE` = тот же sha** — ошибки в Sentry (ЛР5) привязываются к конкретному деплою;
- **`concurrency`** — деплои не идут параллельно и не прерывают друг друга, а встают в очередь;
- **`environment: production`** — в GitHub видна история деплоев и ссылка на приложение, можно включить ручное подтверждение (required reviewers);
- **секреты передаются через `env`**, а не подстановкой `${{ secrets.* }}` в текст скрипта, что исключает инъекцию в shell; `known_hosts` можно задать секретом, иначе используется `ssh-keyscan`;
- на сервере деплой выполняет пользователь `deploy` без root-прав: группа `docker` и `sudo` только на проверку и перезагрузку nginx (настраивается `bootstrap.sh`).

2.5 Настройка репозитория

Таблица 2. Secrets и variables репозитория

Тип	Имя	Значение
secret	`SSH_HOST`	`13.143.182.131`
secret	`SSH_USER`	`deploy`
secret	`SSH_PRIVATE_KEY`	приватный ключ, публичная часть — в `authorized_keys` пользователя deploy
secret	`SSH_KNOWN_HOSTS`	(опц.) вывод `ssh-keyscan <IP>`
secret	`SSH_PORT`	(опц.) порт SSH, по умолчанию 22
variable	`PUBLIC_URL`	`http://13.143.182.131` — для smoke-теста и ссылки environment
variable	`DEPLOY_PATH`	(опц.) по умолчанию `/opt/rental`
variable	`NGINX_SERVER_NAME`	(опц.) если задан — CI обновляет конфиг nginx

```
# отдельный ключ для CI; публичная часть попадает к пользователю deploy
через bootstrap.sh
ssh-keygen -t ed25519 -f ~/.ssh/rental_ci -N "" -C "github-actions-rental-
deploy"
ssh root@13.143.182.131 "SKIP_UPGRADE=true SSH_PUBKEY='$(cat ~/.ssh/
rental_ci.pub)' \
  bash /opt/rental/deploy/scripts/bootstrap.sh"

R=ZZISST/ITMO-ACS-Backend-2026-A
gh secret set SSH_PRIVATE_KEY --repo $R < ~/.ssh/rental_ci
gh secret set SSH_HOST --repo $R --body 13.143.182.131
gh secret set SSH_USER --repo $R --body deploy
grep '^13.143.182.131 ssh-ed25519' ~/.ssh/known_hosts | gh secret set
SSH_KNOWN_HOSTS --repo $R
gh variable set PUBLIC_URL --repo $R --body http://13.143.182.131

# деплой
git push origin HEAD:deploy
```

`.env` на сервере создаётся один раз при первом ручном деплое (ЛР4) — секреты в git не хранятся.

Workflow проверен линтером `actionlint` 1.7.12 (синтаксис, выражения и контексты).

2.6 Результат запуска

Первый push в ветку `deploy` запустил workflow; все 7 job'ов завершились успешно за 6 мин 56 с (test ≈ 1–1.5 мин параллельно, build 2 мин 33 с, deploy 1 мин 54 с).

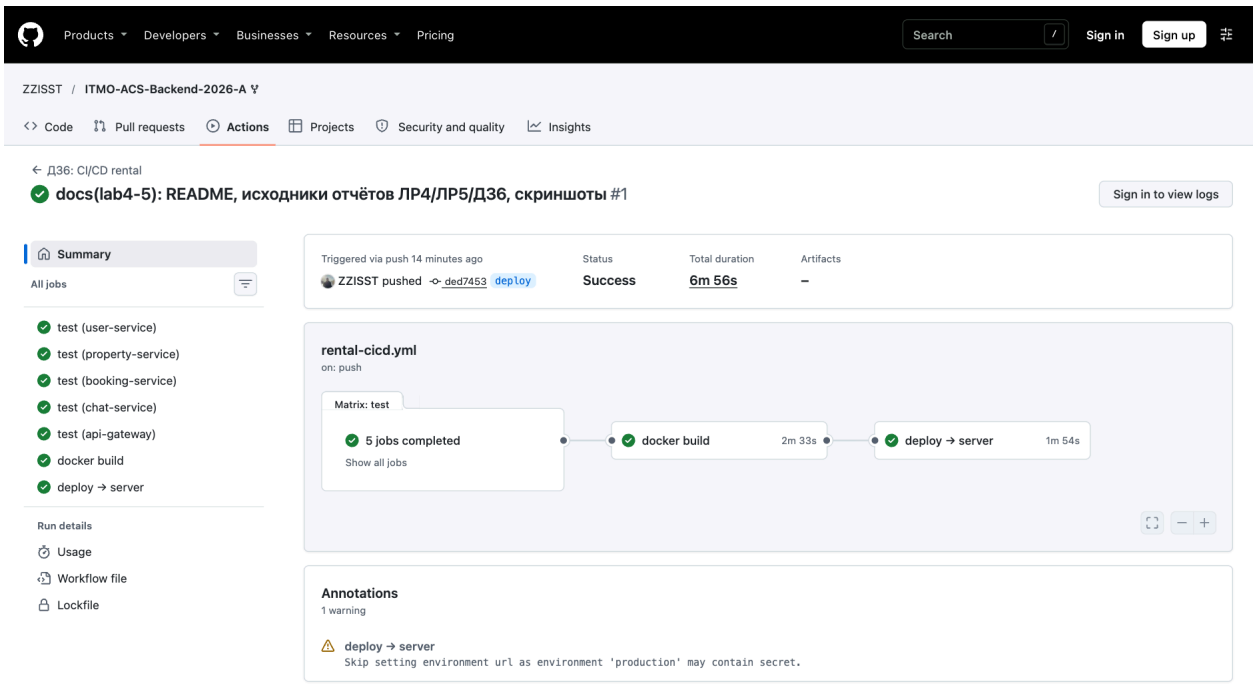


Рисунок 1. Запуск workflow: matrix test ×5 → docker build → deploy → server

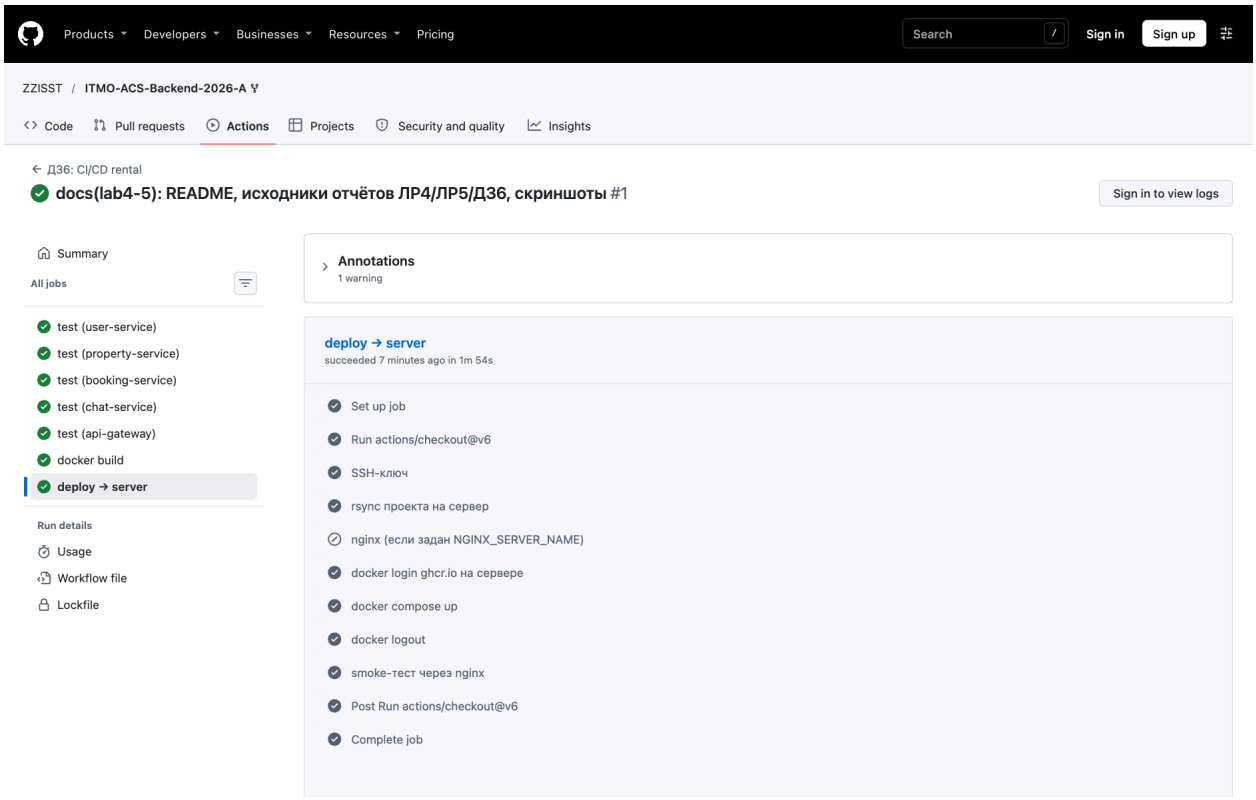


Рисунок 2. Шаги job’а deploy; шаг nginx пропущен, так как NGINX_SERVER_NAME не задан

Фрагмент лога шага «docker compose up» (`deploy.sh` на сервере в режиме `pull`):

```
Login Succeeded
==> mode=pull image=ghcr.io/zzisst/rental-*:ded74532c3fe
release=ded74532c3fe
==> запуск
==> ожидание healthcheck'ов
rental_api_gateway        Up 6 seconds (healthy)
rental_booking_service    Up 12 seconds (healthy)
rental_chat_service       Up 18 seconds (healthy)
rental_property_service   Up 18 seconds (healthy)
rental_user_service       Up 18 seconds (healthy)
{"service":"api-gateway","status":"ok"}
==> очистка старых образов
==> *** OK
Removing login credentials for ghcr.io
{"service":"api-gateway","status":"ok"}Деплой ded74532c3fe доступен:
http://***
```

После деплоя на сервере работают образы из GHCR:

```
$ docker compose ps --format "{{.Name}} {{.Image}} {{.Status}}" | grep
rental_
rental_api_gateway ghcr.io/zzisst/rental-api-gateway:ded74532c3fe Up 47
seconds (healthy)
rental_booking_service ghcr.io/zzisst/rental-booking-service:ded74532c3fe
Up 53 seconds (healthy)
rental_chat_service ghcr.io/zzisst/rental-chat-service:ded74532c3fe Up
About a minute (healthy)
rental_property_service ghcr.io/zzisst/rental-property-service:ded74532c3fe
Up About a minute (healthy)
rental_user_service ghcr.io/zzisst/rental-user-service:ded74532c3fe Up
About a minute (healthy)
```

GitHub маскирует в логах значения секретов, поэтому слово `deploy` (значение `SSH_USER`) и IP (значение `SSH_HOST`, входит в `PUBLIC_URL`) заменены на ***, а ссылка environment не выставляется (предупреждение в Annotations). Это не ошибка; если нужно видеть их в логах, `SSH_HOST` и `SSH_USER` можно хранить как variables — они не являются секретами.

2.7 Проблема при повторном деплое и её решение

Следующий запуск упал на шаге «push в GHCR» job'a build (deploy при этом не выполнялся, на сервере осталась предыдущая версия):

```
Pushing ghcr.io/zzisst/rental-api-gateway:84759f47abb1: 08bc4e534116 Layer
already exists
```

```
denied: permission_denied: write_package
##[error]Process completed with exit code 1.
```

Причина. Первый запуск создал пакеты `rental-*` в GHCR, но они не были связаны с репозиторием. `GITHUB_TOKEN` с правом `packages: write` позволяет создать новый пакет, а перезаписать существующий — только если пакет связан с репозиторием, из которого запущен workflow (или репозиторию явно выдан доступ в настройках пакета).

Решение. В финальный stage всех Dockerfile добавлена OCI-метка, по которой GHCR автоматически связывает пакет с репозиторием и выдаёт workflow права на запись:

```
LABEL org.opencontainers.image.source="https://github.com/ZZISST/ITMO-ACS-Backend-2026-A" \
      org.opencontainers.image.title="rental-user-service"
```

Так как старые пакеты уже были созданы без связи, образы стали публиковаться под новым именем `ghcr.io/zzisst/backend-rental-<сервис>`. Проверка: запуск после исправления прошёл успешно, после чего тот же запуск был перезапущен (`gh run rerun`) — повторная публикация образов с тем же тегом в уже существующие пакеты также прошла, и сервер работает на новых образах:

```
$ gh run view 41822783569 --attempt 2
✓ test (booking-service) in 23s     ✓ test (chat-service) in 20s
✓ test (property-service) in 24s    ✓ test (user-service) in 21s
✓ test (api-gateway) in 20s
✓ docker build in 2m44s
✓ deploy → server in 2m2s

$ docker compose ps --format "{{.Name}} {{.Image}} {{.Status}}"   # на сервере
rental_api_gateway ghcr.io/zzisst/backend-rental-api-gateway:0039dcc21970 Up (healthy)
rental_booking_service ghcr.io/zzisst/backend-rental-booking-service:0039dcc21970 Up (healthy)
rental_chat_service ghcr.io/zzisst/backend-rental-chat-service:0039dcc21970 Up (healthy)
rental_property_service ghcr.io/zzisst/backend-rental-property-service:0039dcc21970 Up (healthy)
rental_user_service ghcr.io/zzisst/backend-rental-user-service:0039dcc21970 Up (healthy)
```

3 Вывод

Настроен пайплайн GitHub Actions, который при каждом push в ветку `deploy` проверяет код всех микросервисов (форматирование, статический анализ, тесты с race detector, сборка), собирает Docker-образы и доставляет приложение на сервер по SSH, после чего убеждается, что все контейнеры здоровы и приложение отвечает через nginx. Pull request'ы проходят только проверки без деплоя. Деплой использует тот же скрипт, что и ручное развёртывание из ЛР4, выполняется непривилегированным пользователем, а версия релиза передаётся в Sentry.